

CSCI 5523 Introduction to Data Mining (Spring 2026)

Assignment 2 (10 points)

Deadline: March 4th, 11:59 PM CDT (Gradescope)

1. Overview of the Assignment

In this assignment, you will implement the **SON algorithm** using the Apache Spark Framework. You will develop a program to identify frequent itemsets in two datasets: one simulated (Task 1) and one real-world (Task 2), generated from the Yelp dataset. The goal of this assignment is to apply the algorithms you have learned in class on large datasets more efficiently in a distributed environment.

2. Requirements

2.1 Programming Requirements

- a. You must use **Python** to implement all tasks. You can only use standard Python libraries (i.e., external libraries such as NumPy or pandas are not allowed).
- b. **You are required only to use Spark RDD, i.e.,** no point in using Spark DataFrame or DataSet.

2.2 Submission Platform

We will use a submission platform to run and grade your submission automatically. We highly recommend testing your scripts on your local machine before submitting your solutions. **We will use Gradescope to grade your submissions.**

2.3 Programming Environment

Python 3.9.12, and Spark 3.2.1 or Spark 3.5.4

2.4 Write your own code

Do not use large language models (e.g., ChatGPT, Copilot) or share code with other students!

For this assignment to be an effective learning experience, **you must write your own code**. We emphasize this point because you can find Python implementations of some of the required functions online. Please do not look for or at any such code.

TAs will combine all code we can find on the web (e.g., GitHub), other students' code from this and other (previous) sections, and code generated by AI tools for plagiarism detection. **We will report all detected plagiarism to the university.**

3. Datasets

In this assignment, you will implement the algorithms using simulated and real-world datasets.

In Task 1, you will build and test your program with two simulated CSV files (small1.csv and small2.csv).

In Task 2, you will first generate a subset from the Yelp dataset using business.json and review.json (see Section 4.2.1). You will build and test your program with this real-world dataset **locally** (you do not need to submit the dataset). **TAs will use different sampled subsets of the Yelp datasets for grading.**

Figure 1 shows an example of the file format, where the first column is “year”, the second column is “user_id”, the third column is “business_id”, and the fourth column is “rate”.

year	user_id	business_id	rate
2016	3	16	0
2016	1	16	4
2016	25	6	3
2018	16	23	2
2016	15	5	2
2016	46	13	5

Figure 1: Input Data Format

4. Tasks

The goal of this assignment is to find all the possible combinations of the frequent itemsets in a given input file within the required time. You will implement the **SON algorithm** to solve Task 1 and Task 2. In the first pass of SON, you will process each data partition using **the A-Priori algorithm for Task 1 and the PCY algorithm for Task 2**. Complete the provided skeleton code in task1.py and task2.py. You can write your own helper functions to support the tasks.

4.0 Submission

You need to submit the following files on Gradescope:

- (Required) Two Python scripts (all lowercase): **task1.py, task2.py**
- (Optional) Other Python scripts containing functions

4.1 Task 1: Simulated data (5 pts)

There are two CSV files (small1.csv and small2.csv) provided in the data folder. The small1.csv file is a sample you can use to debug your code. **TAs will grade your code with small2.csv.**

4.1.1 Task Description

You will implement two cases (argument: c) of market-basket models:

- **Case 1:** Calculate frequent businesses
- **Case 2:** Calculate frequent users

For each case, you need to find all frequent itemsets, including singletons, pairs, triples, etc., based on a given support threshold (argument: t) in a specified year (argument: y). To achieve this, you will use the **SON algorithm** by applying the **A-Priori algorithm to each partition of the data in the first pass** to generate candidate itemsets, and then counting the occurrences of these candidates across the entire dataset in the second pass to identify globally frequent itemsets.

Case 1 (2.5 pts):

You will create a basket for each user that contains the business IDs reviewed by that user in the specified year. If a business was reviewed more than once by a reviewer in that year, we consider the business rated only once; i.e., the business IDs within each basket are unique. The generated baskets will look like:

```
user1: [business11, business12, business13, ...]
user2: [business21, business22, business23, ...]
user3: [business31, business32, business33, ...]
```

Case 2 (2.5 pts):

You will create a basket for each business, containing the user IDs of users who reviewed it in the specified year. As in case 1, the user IDs within each basket are unique. The baskets will be like:

```
business1: [user11, user12, user13, ...]
business2: [user21, user22, user23, ...]
business3: [user31, user32, user33, ...]
```

4.1.2 Input Format (Please make sure you use exactly the same input parameter names!)

Similar to HW1, you will use the “**argparse**” module to parse the following arguments:

1. Year used (**--y**): Integer that specifies what year is used for the analysis
2. Rate used (**--r**): Integer that specifies which rate is filtered
2. Case number (**--c**): Integer that specifies the case: **1 for Case 1, and 2 for Case 2.**
2. Threshold (**--t**): Integer that defines the support threshold to qualify as a frequent itemset.
3. Input file path (**--input_file**): Path to the input file, including path, file name, and extension.
4. Output file path (**--output_file**): Path to the output file, including path, file name, and extension.

Execution example:

```
$ python task1.py --y <year> --r <rate> --c <case number> --t <minimum count> --input_file
<input_file_path> --output_file <output_file_path>
```

Example: \$ python task1.py --y 2016 -r 0 --c 1 --t 7 --input_file ./data/small1.csv --output_file ./results/task1_output.json

4.1.3 Output Format

You must output the results, including (1) the candidates of frequent itemsets, (2) the final frequent itemsets, (3) the number of candidates by size with (4) total, and (5) runtime (seconds), and, in **one** JSON file (.json).

(1) Candidates: The list contains lists of singletons, pairs, triples, etc., where the itemsets are sorted in **lexicographical** order. Both user_id and business_id have the data type "string". You should use "**Candidates**" as the tag.

(2) Frequent Itemsets: You need to output the **final frequent itemsets** along with their support. You should use "**Frequent Itemsets**" as the tag.

(3) Num Candidates By Size: A dictionary that contains the number of candidate itemsets for each size.

- Key: itemset k (e.g., 1 for singletons, 2 for pairs, 3 for triples, ...)
- Value: total count of candidate itemsets of size K

Keys are integers and are sorted in increasing order when written to JSON. You should use "**Num Candidates By Size**" as the tag.

(4) Num Candidates Total: An integer equal to the total number of candidate itemsets across all sizes, i.e., the sum of counts in Num Candidates By Size. You should use "**Num Candidates Total**" as the tag.

(5) Runtime Duration: You need to save the runtime with a "**Runtime**" tag: "Runtime": <time_in_seconds>, e.g., "Runtime": 100.00. The total execution time (seconds) from loading the file till finishing writing the output file. **The provided code includes specific lines for measuring execution time.**

Here is an example of the output file format:

```
{
  "Candidates": [[["13"], ["16"], ["6"], ["8"]], [["13", "6"]]],
  "Frequent Itemsets": [[["13"], ["16"], ["6"], ["8"]], [["13", "6"]]],
  "Num Candidates By Size": {"1": 4, "2": 1},
  "Num Candidates Total": 5,
  "Runtime": 4.517187118530273
}
```

Figure 2: Task 1 Example Output

4.2 Task 2: Yelp data (5 pts)

4.2.1 Task Description

In Task 2, you will explore the Yelp dataset to identify frequent business sets (**Task 1, Case 1**). You will use the business.json and review.json provided in the data folder to generate a CSV file for local tests.

Data preprocessing (0 pts)

You will generate a sample dataset from business.json and review.json with the following steps:

1. Filter the businesses in the state “Arizona” using business.json, i.e., “state”== “AZ”.
2. Select (user_id, business_id) from review.json whose business_id is from Arizona.
3. Generate a CSV file with the columns “user_id” and “business_id”. Figure 3 shows an example of the output file.

user_id	business_id
hG7b0MtEbXx5QzbzE6C_VA	ujmEBvifdJM6h6RLv4wQlg
yXQM5uF2jS6es16SJzNHfg	NZnhc2sEQy3RmzKTZnqtwQ
nMeCE5-xsdleyxYuNZ_7rA	oxwGyA17NL6c5t1Etg5WgQ
Flk4lQQu1eTe2EpzQ4xhBA	8mlrX_LrOnAqWsB5JrOojQ

Figure 3: user_business file

You DO NOT need to submit the code or data for this step. TAs will use different filtering criteria to generate datasets for grading.

Apply the SON algorithm combined with the PCY algorithm (5 pts)

You will find the frequent business sets (**Task 1 Case 1**) with the large dataset you generated. To achieve this, you will use the **SON algorithm** by applying the **PCY algorithm to each partition of the data in the first pass** to generate candidate itemsets, and then counting the occurrences of these candidates across the entire dataset in the second pass to identify globally frequent itemsets (Hint: Use a hash table to hash the pairs into hash buckets).

You need to do the following steps:

1. Load the preprocessed user_business CSV file and build Case 1 market-basket model
2. Filter qualified users who reviewed more than **b unique** businesses (argument: *b*)
3. Apply the SON algorithm combined with the PCY algorithm to the market-basket model. Hints are provided with comments in the code skeleton file - task2.py

4.2.2 Input format (Please make sure you use the same input parameter names!)

Similar to HW1, you will use the “argparse” module to parse the following arguments:

1. Filter threshold (--b): An integer that is used to keep only qualified users who reviewed **more than b** distinct businesses.
2. Threshold (--t): Integer that defines the support threshold to qualify as a frequent itemset.
3. Input file path (--input_file): Path to the input file, including path, file name, and extension. **This is the preprocessed user_business CSV file you generated.**

4. Output file path (**--output_file**): Path to the output file, including path, file name, and extension.

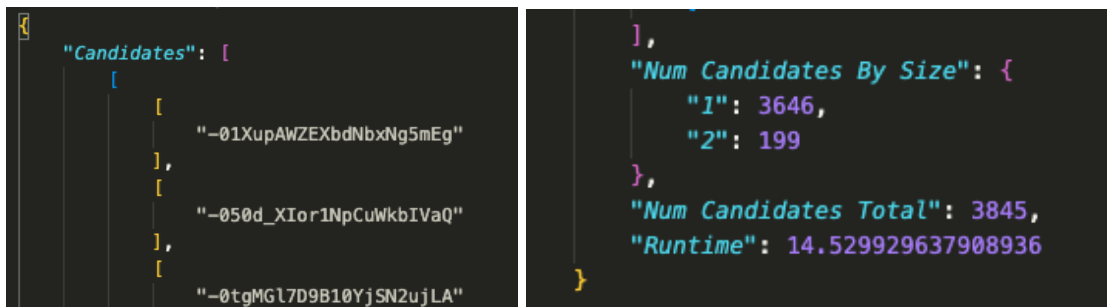
Execution example:

```
$ python task2.py --b <filter users> --t <threshold> --input_file <input_file_path> --output_file <output_file_path>
```

Example: `$ python task2.py --b 4 --t 7 --input_file ./data/user_business.csv --output_file ./results/task2_output.json`

4.2.3 Output Format

You must write the results, including the candidates of frequent itemsets, the final frequent itemset, and runtime, in **one** JSON file (.json). **The output format is the same as Section 4.1.3.**



```
{
  "Candidates": [
    [
      "-01XupAwZEXbdNbxNg5mEg"
    ],
    [
      "-050d_XIor1NpCuWkbIVaQ"
    ],
    [
      "-0tgMGl7D9B10YjSN2ujLA"
    ]
  ],
  "Num Candidates By Size": {
    "1": 3646,
    "2": 199
  },
  "Num Candidates Total": 3845,
  "Runtime": 14.529929637908936
}
```

Figure 4: Task 2 Example Output (the full results are lengthy, so only selected key–value pairs are shown).

5. Evaluation Metric

Please refer to the following metrics to determine whether your implementation is efficient enough. We will terminate your code on a test dataset if the running time is extremely long. Correct implementations with execution time below these requirements will get full points. Slower execution times that output correct results will get half points.

Input File	Case(c)	Threshold (t)	Year (y)	Rate (r)	Runtime (sec)
small2.csv	1	4	2016	0	<=20
small2.csv	2	9	2016	0	<=14

Input File	Filter Threshold (f)	Threshold (t)	Runtime (sec)
user_business.csv	5	5	<=75

6. Grading Criteria

(% penalty = % penalty of possible points you get)

1. You can use your free 5-day extension separately or together. During grading, TAs will count late days **based on the time of submission**. For example, if the due date is 2026/02/24 23:59:59 CDT and your submission is 2026/02/25 05:23:54 CDT, the number of late days would be 1. **TAs will record grace days by default (NO separate Emails)**. However, if you want to save it for next time and treat your assignment as a late submission (with some penalties), **please leave a comment in your submission**.
2. There will be no point if your submission cannot be executed on the grading platform.
3. There is no regrading. Once the grade is posted on Canvas, we will only regrade your assignments if there is a grading error. No exceptions.
4. Homework assignments are due at 11:59 pm CT on the due date and should be submitted on Canvas. Late submissions within 24 hours of the due date will receive a 30% penalty. Submissions made more than 24 hours after the due date will receive a 50% penalty. Every student has FIVE free late days for homework assignments. You can use these free late days for any reason, separately or together, to avoid the late penalty. There will be no other extensions for any reason. **You cannot use the free late days after the last day of the class.**

7. Submission Instructions

1. You will submit your programming assignment via Gradescope. You can access it from Canvas. The Gradescope is available now.
2. On Gradescope, you will find a link for the submission of **TWO** Python files (lowercase):
task1.py
task2.py
3. You **MUST** follow the parameter naming convention (case-sensitive) provided in the description of each task (see the execution examples for each task). Failure to do this will result in failing all test cases (even if your code is correct). It is your responsibility to make sure that your filenames strictly follow the instructions above.
4. Once you submit your programming files, the grading scripts on Gradescope will take some time to run your files and test cases. You should be able to see the final score for each task. **However, you should NOT use Gradescope as a debugging tool.** You should first debug your code on your local machine before submitting your code to Gradescope!
5. On Gradescope, there is a function named `test_tasklog` to display the stdout and stderr when running your code on Gradescope.

```
Stdout when running your task files
logs for task1cl
Setting default log level to "WARN".
```

```
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
```

```
23/03/08 19:20:08 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

```
[Stage 0:>
```

```
(0 + 1) / 1]
```

6. If you did not pass a test case for a particular task, this means that there is an error (errors) that you have to fix. On the other hand, passing all test cases is a good sign, **but it does not mean that you will pass the test cases that the TA will use for grading**. Your final grade is ultimately determined by the test cases that are run after the due date (i.e., not the test cases that are available to students before the due date).
7. You may submit your code to Gradescope as long as it is submitted before the due date. If you submit your code after the due date, it will be recorded as a late submission (See Section 6 for information on grading criteria).